

SWE 4101

Object Oriented Concepts I

Objectives

01

Type Promotion

02

Conditional
Statements

03

Loops

04

Arrays

05

ArrayList

06

Encapsulation



✓ Integer Division

```
int i = 10 / 4;
```

```
System.out.println(i); // Prints 2
```

When we apply the division operator on two integers like a/b , it always returns an integer, even though a isn't evenly divisible by b

Integer Division

```
float x = 10 / 4;
```

```
System.out.println(x); // Prints 2.0
```

Even if we assign the calculation result to a float variable, the result is still 2.0

But why?

- ✓ • Java Specification states that integer division rounds toward zero
- ✓ • The result of the division operation of two integers is only the quotient value
- ✓ • The remainder is not included

But why?

```
int x = -10 / 4;
```

```
System.out.println(x); // Prints -2
```

- ✓ The floor of -2.5 is -3
- ✓ But Java rounds towards 0
- ✓ Hence, we get -2

✓ The Solution: Type Promotion

- ✓ ● How do we make an integer division result in float?
- ✓ ● Java enables us to use Type Promotion -> a form of Implicit Typecasting
- ✓ ● When different primitive types are used in an expression, Java promotes operands to the higher type in the numeric promotion hierarchy before performing the operation

✓ Type Promotion

Java promotion rules for mixed arithmetic operators.

If two rules apply, choose the one that occurs first in this table.

If either operand is	The other is promoted to
double	double
float	float
long	long
byte or short	int

double
↑
float
↑
long
↑
int
↑
byte, short, char

✓ Type Promotion

```
float x = (float) 10 / 4;  
System.out.println(x); // Prints 2.5
```

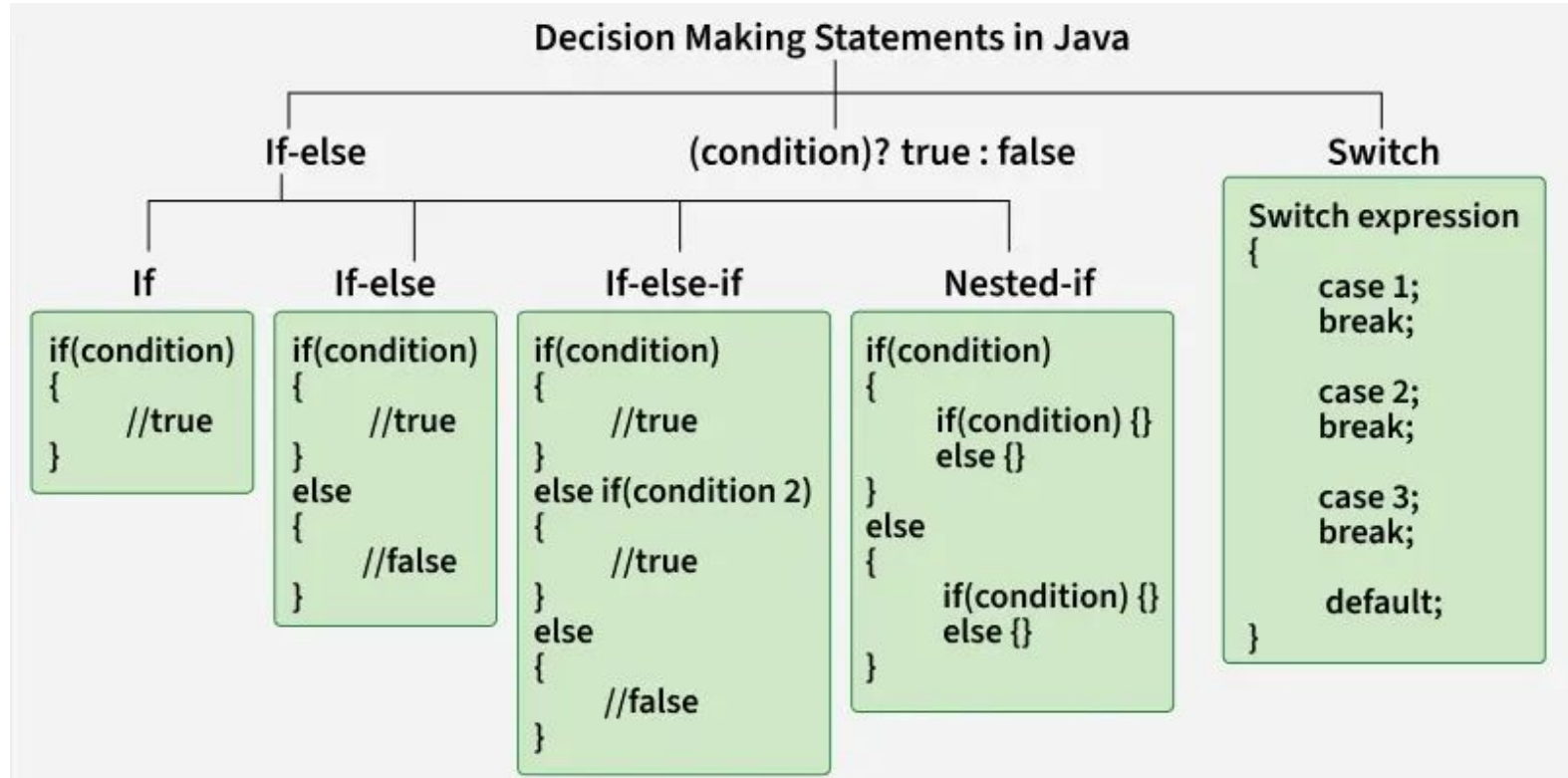
```
float y = 10 / (float) 4;  
System.out.println(x); // Prints 2.5
```

Casting one operand to float automatically promotes the other to float

Type Promotion Example

```
void main() {  
  
    byte b = 10;  
    short s = 20;  
    char c = 'A'; // ASCII 65  
    int i = 30;  
  
    // All promoted to int before operation  
    int result1 = b + s + c + i;  
  
    // long + float → promoted to float  
    float result2 = 100L + 5.5f;  
  
    System.out.println(result1); // 125  
    System.out.println(result2); // 105.5  
}
```

✓ Java Conditional Statements



✓ If Statement

```
class Geeks {  
    public static void main(String args[])  
    {  
        int i = 10;  
  
        if (i < 15) {  
            System.out.println("Condition is True");  
        }  
    }  
}
```

✓ If-else Statement

```
class Geeks {  
    public static void main(String args[])  
    {  
        int i = 10;  
  
        if (i < 15)  
            System.out.println("i is smaller than 15");  
        else  
            System.out.println("i is greater than 15");  
    }  
}
```

✓Nested-if Statement

```
class Geeks {  
    public static void main(String args[])  
    {  
        int i = 10;  
  
        // Outer if statement  
        if (i < 15) {  
            System.out.println("i is smaller than 15");  
  
            // Nested if statement  
            if (i == 10) {  
                System.out.println("i is exactly 10");  
            }  
        }  
    }  
}
```

✓ If-else-if Statement

```
class Geeks {  
    public static void main(String args[])  
    {  
        int i = 20;  
  
        if (i == 10)  
            System.out.println("i is 10");  
        else if (i == 15)  
            System.out.println("i is 15");  
        else if (i == 20)  
            System.out.println("i is 20");  
        else  
            System.out.println("i is not present");  
    }  
}
```

✓ Switch case

```
class Geeks {  
    public static void main(String[] args)  
    {  
        int num = 20;  
        switch (num) {  
            case 10:  
                System.out.println("It is 5");  
                break;  
            case 20:  
                System.out.println("It is 20");  
                break;  
            default:  
                System.out.println("Not present");  
        }  
    }  
}
```

✓ Ternary Operator

A conditional operator that provides a shorthand way to write simple if-else statements

```
class Geeks {  
    public static void main(String args[]) {  
        int a = 10, b = 20;  
        int max = (a > b) ? a : b;  
  
        System.out.println("Maximum is " + max);  
    }  
}
```

✓ Java Loops: For Loop

Ideal for situations where the number of iterations is known beforehand

```
class Geeks {  
    public static void main(String[] args)  
    {  
        for (int i = 0; i <= 10; i++) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

✓ For Each Loop

- ✓ • Used to iterate over arrays or collections in a more readable and concise manner
- ✓ • No need for an explicit counter or index variable

```
for (dataType item : array) {  
    // Code to be executed  
}
```

✓ For Each Loop

- ✓ Used to iterate over arrays or collections in a more readable and concise manner
- ✓ No need for an explicit counter or index variable

```
public class demo {  
    public static void main(String[] args) {  
  
        int[] numbers = {10, 20, 30, 40, 50};  
  
        for (int number : numbers) {  
            System.out.println("Number: " + number);  
        }  
    }  
}
```

✓ While Loop

It continues to execute the code block as long as the specified condition remains true

```
class Geeks {  
    public static void main(String[] args)  
    {  
        int i = 0;  
        while (i <= 10) {  
            System.out.print(i + " ");  
            i++;  
        }  
    }  
}
```

✓ Do-while Loop

Similar to the while loop but ensures that the code block is executed at least once before the condition is checked

```
class Geeks {  
    public static void main(String[] args)  
    {  
        int i = 0;  
        do {  
            System.out.print(i + " ");  
            i++;  
        } while (i <= 10);  
    }  
}
```

✓ Arrays

- ✓ ● Objects that store multiple values of the same data type in a single variable
- ✓ ● Can hold both primitive types (like int, char, boolean, etc.) and objects
- ✓ ● Arrays of primitive types store the elements in contiguous locations
- ✓ ● For non primitive types, references to items are stored at contiguous locations
- ✓ ● Fixed size

✓ Declaring Arrays

Syntax:

`dataType[] arrayName;`

or

`dataType arrayName[];` (C-Style declaration)

```
int[] num;
```

```
int num[];
```

✓ Initializing Arrays

- ✓ Declaring an array only creates a reference
- ✓ To allocate memory, we initialize it

```
int[] num = new int[3];
```

```
int[] num = {1, 2, 3};
```

- ✓ The elements in the array allocated by new gets default values
- ✓ If you already know the values, you don't need to write new

✓ Length of an Array

To find the size of array java provides a built-in property called length

```
public static void main(String[] args){  
    int[] arr = {2, 4, 8, 12, 16};  
    System.out.println("Size of array: " + arr.length);  
}
```

✓ Accessing Elements

- ✓ Elements of an array can be accessed by their index
- ✓ Array indexing starts from 0

```
public static void main(String[] args){  
  
    int[] arr = {2, 4, 8, 12, 16};  
  
    // Accessing fourth element  
    System.out.print(arr[3] + " ");  
  
    // Accessing first element  
    System.out.print(arr[0]);  
}
```

✓ Traversing an Array

- ✓ Can be traversed using any form of loop

```
public static void main(String[] args) {  
  
    int[] arr = {2, 4, 8, 12, 16};  
  
    // Traversing the array using a for loop  
    for(int i = 0; i < arr.length; i++) {  
  
        System.out.print(arr[i] + " ");  
    }  
}
```

✓ Multidimensional (2D) Arrays

// Declaration and Initialization

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

// Traversing the 2D array

```
for(int i = 0; i < matrix.length; i++) {  
    for(int j = 0; j < matrix[i].length; j++) {  
        System.out.print(matrix[i][j] + " ");  
    }  
    System.out.println();  
}
```

✓ ArrayList

- ✓ A resizable array provided in the java.util package
- ✓ Unlike normal arrays, its size can grow or shrink dynamically as elements are added or removed
- ✓ Elements can be accessed using their indices, similar to arrays
- ✓ Duplicates are allowed
- ✓ Elements are stored in the order they are inserted

✓ ArrayList

```
import java.util.ArrayList;

class Main {

    public static void main (String[] args) {

        ArrayList<Integer> a = new ArrayList<>(); // Creating an ArrayList

        a.add(1); // Adding Element in ArrayList

        a.add(2);

        a.add(3);

        System.out.println(a); // Prints [1, 2, 3]

    }

}
```



ArrayList doesn't store primitive data types directly. Rather it stores the object versions of those primitive data types.

ArrayList<Integer> a = new ArrayList<>(); -> Correct

ArrayList<int> a = new ArrayList<>(); -> Wrong

Wrapper Classes for ArrayList

- ✓ ArrayList stores objects
- ✓ But int, double, float, etc. are primitives, not objects
- ✓ Java provides wrapper classes
- ✓ They are object versions of primitive data types

Primitive data types	Wrapper class
boolean	Boolean
<u>char</u>	<u>Character</u>
byte	Byte
short	Short
<u>int</u>	<u>Integer</u>
long	Long
float	Float
double	Double

✓ Creating an ArrayList

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList
    }
}
```

✓ Adding an element

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList

        fruits.add("Apple"); // Adding elements
        fruits.add("Banana");
    }
}
```

✓ Adding at a specific index

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList

        fruits.add("Apple"); // Adding elements
        fruits.add("Banana");
        fruits.add(1, "Orange"); // Adding element at a specific index
    }
}
```

✓ Accessing Elements

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList

        fruits.add("Apple"); // Adding elements
        fruits.add("Banana");
        fruits.add(1, "Orange"); // Adding element at a specific index
        fruits.get(0); // Accessing using an index
    }
}
```

✓ Removing by index

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList

        fruits.add("Apple"); // Adding elements
        fruits.add("Banana");
        fruits.add(1, "Orange"); // Adding element at a specific index
        fruits.get(0); // Accessing using an index
        fruits.remove(0); // Removing element using index
    }
}
```

✓ Removing by value

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList

        fruits.add("Apple"); // Adding elements
        fruits.add("Banana");
        fruits.add(1, "Orange"); // Adding element at a specific index
        fruits.get(0); // Accessing using an index
        fruits.remove(0); // Removing element using index
        fruits.remove("Banana"); // Removing element using value
    }
}
```

✓ Updating an index

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList

        fruits.add("Apple"); // Adding elements
        fruits.add("Banana");
        fruits.add(1, "Orange"); // Adding element at a specific index
        fruits.get(0); // Accessing using an index
        fruits.remove(0); // Removing element using index
        fruits.remove("Banana"); // Removing element using value
        fruits.set(0, "Mango"); // Updating element
    }
}
```

✓ Printing an ArrayList

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>(); // Creating an ArrayList

        fruits.add("Apple"); // Adding elements
        fruits.add("Banana");
        fruits.add(1, "Orange"); // Adding element at a specific index
        fruits.get(0); // Accessing using an index
        fruits.remove(0); // Removing element using index
        fruits.remove("Banana"); // Removing element using value
        fruits.set(0, "Mango"); // Updating element

        System.out.println("After updating value: " + fruits);
    }
}
```

✓ Printing an ArrayList

`fruits.size();` *// Size of ArrayList*

`fruits.isEmpty();` *// Checks whether empty*

`fruits.contains("Apple");` *// Checks if element exists*

`fruits.clear();` *// Clear the ArrayList*

`fruits.indexOf("Apple");` *// Index of the first occurrence*

`fruits.lastIndexOf("Apple");` *// Last occurrence*

✓ Encapsulation

4 pillars of OOP:
i. Encapsulation
ii. Polymorphism
iii. Inheritance
iv. Abstraction

- ✓ ● Encapsulation is one of the four pillars of OOP
- ✓ ● A means of protecting the class, the integrity of its data and behavior
- ✓ ● Bundles data (variables) and methods (functions) into a single unit, such as a class, while **restricting direct access to some of an object's components**

✓ A Banking Scenario

- ✓ ● Consider a small retail bank that needs a software model for a customer's account
- ✓ ● The bank's first business rules are:
 - ✓ 1. Every account has a unique account number assigned by the bank, never by the customer
 - ✓ 2. Every account must have an owner and a currency at the moment it is opened
 - ✓ 3. The balance of any account must never be negative

A Banking Scenario

```
public class BankAccount {  
    public String accountNumber;  
    public String ownerName;  
    public double balance;  
    public String currency;  
}
```

✓ A Banking Scenario

- ✓ 4. Can deposit or withdraw money but the amount should be positive

```
public class BankAccount {  
    public String accountNumber;  
    public String ownerName;  
    public double balance;  
    public String currency;  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount)  
    {  
        balance = balance - amount;  
    }  
}
```

A Banking Scenario

Before making EVERYTHING private, ask these questions:

- ✓ Which fields, if any, should the outside world be able to modify directly?
- ✓ Which should it be able to read but not modify?
- ✓ Which should it not be able to see at all?

✓ Making fields 'private'

```
public class BankAccount {  
    private String accountNumber;  
    private String ownerName;  
    private double balance;  
    private String currency;  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount)  
    {  
        balance = balance - amount;  
    }  
}
```